MONITORING TOOLS & OPERATIONS GUIDE

AI-Based Monitoring Solution for Oil & Gas Industry

Document Version: 1.0 **Date**: December 2025 **Audience**: DevOps Engineers, SREs, Platform Engineers

---

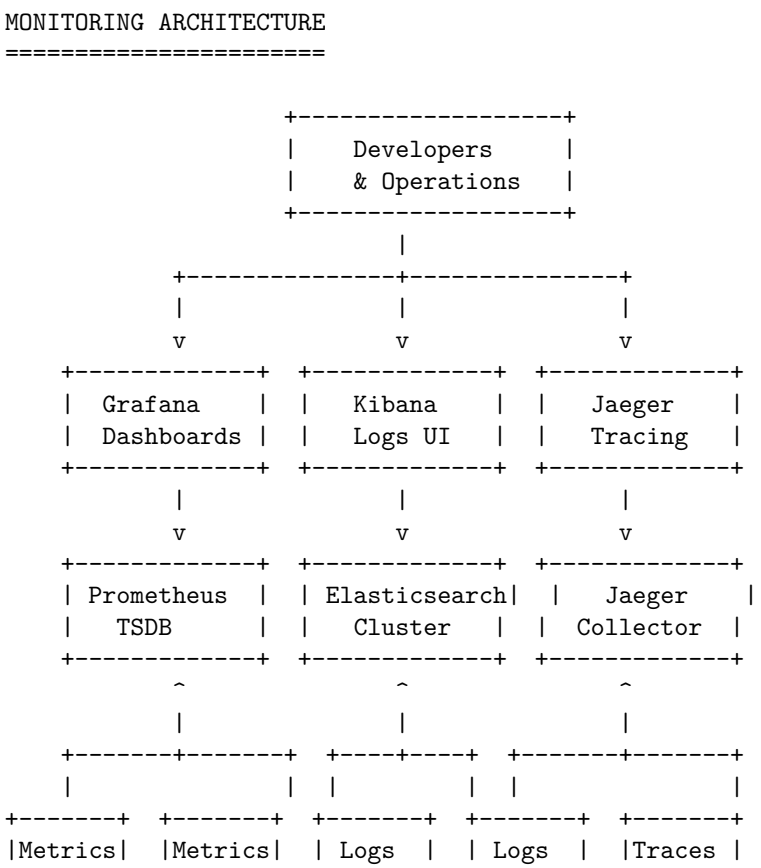
TABLE OF CONTENTS

1. Monitoring Architecture Overview
2. Prometheus Operations
3. Grafana Operations
4. ELK Stack Operations
5. Alertmanager Configuration
6. Jaeger Tracing
7. Application Metrics
8. Log Management
9. Dashboards & Visualization
10. Complete Monitoring Workflows

---

1. MONITORING ARCHITECTURE OVERVIEW

1.1 Monitoring Stack



```
|Exporter| |Exporter| |Shipper|  |Shipper|  |  SDK  |
+-------+  +-------+  +-------+  +-------+  +-------+
    |          |          |          |          |
+-------+  +-------+  +-------+  +-------+  +-------+
|ML API |  |Backend|  |ML API |  |Backend|  |ML API |
| Pod   |  | Pod   |  | Pod   |  | Pod   |  | Pod   |
+-------+  +-------+  +-------+  +-------+  +-------+
```

1.2 Tool Versions & Installation

```bash
# Prometheus Stack via Helm
helm repo add prometheus-community https://prometheus-community.github.io/helm-charts
helm install prometheus prometheus-community/kube-prometheus-stack -n monitoring

# ELK Stack via Helm
helm repo add elastic https://helm.elastic.co
helm install elasticsearch elastic/elasticsearch -n logging
helm install kibana elastic/kibana -n logging
helm install filebeat elastic/filebeat -n logging

# Jaeger via Helm
helm repo add jaegertracing https://jaegertracing.github.io/helm-charts
helm install jaeger jaegertracing/jaeger -n tracing

# CLI tools
# promtool
wget https://github.com/prometheus/prometheus/releases/download/v2.47.0/prometheus-2.47.0.linux-amd64.ta
tar xzf prometheus-2.47.0.linux-amd64.tar.gz
sudo mv prometheus-2.47.0.linux-amd64/promtool /usr/local/bin/
```

---

2. PROMETHEUS OPERATIONS

2.1 PromQL Queries

```
# ============================================
# BASIC QUERIES
# ============================================

# Instant vector - current CPU usage
node_cpu_seconds_total{mode="idle"}

# Range vector - last 5 minutes
node_cpu_seconds_total{mode="idle"}[5m]

# Filter by labels
http_requests_total{job="ml-api", status="200"}

# Regex match
http_requests_total{job=~"ml-api|backend"}
http_requests_total{path!~"/health|/metrics"}

# ============================================
```

```
# AGGREGATIONS
# ==========================================

# Sum across all instances
sum(http_requests_total)

# Sum by label
sum by (status) (http_requests_total)
sum by (instance, job) (http_requests_total)

# Count instances
count(up{job="ml-api"})

# Average
avg(node_memory_MemAvailable_bytes / node_memory_MemTotal_bytes * 100)

# Min/Max
max(container_memory_usage_bytes{container="ml-api"})
min(container_cpu_usage_seconds_total{container="ml-api"})

# Quantile (percentile)
histogram_quantile(0.95, rate(http_request_duration_seconds_bucket[5m]))

# ==========================================
# RATE & INCREASE
# ==========================================

# Rate - per second average over time range
rate(http_requests_total[5m])

# Increase - total increase over time range
increase(http_requests_total[1h])

# irate - instant rate (more volatile)
irate(http_requests_total[1m])

# ==========================================
# OPERATORS & FUNCTIONS
# ==========================================

# Arithmetic
(node_memory_MemTotal_bytes - node_memory_MemAvailable_bytes) / node_memory_MemTotal_bytes * 100

# Comparison
http_requests_total > 1000
http_request_duration_seconds > 0.5

# Logical
http_requests_total > 100 and http_requests_total < 1000

# Offset - query historical data
http_requests_total offset 1h

# Prediction
```

```
predict_linear(node_filesystem_avail_bytes[1h], 4*3600)

# Changes
changes(up[1h])

# Absent - alert when metric missing
absent(up{job="ml-api"})

# ============================================
# ML API SPECIFIC QUERIES
# ============================================

# Request rate by endpoint
sum by (path) (rate(http_requests_total{job="ml-api"}[5m]))

# Error rate
sum(rate(http_requests_total{job="ml-api",status=~"5.."}[5m]))
/ sum(rate(http_requests_total{job="ml-api"}[5m])) * 100

# P95 latency
histogram_quantile(0.95,
  sum by (le) (rate(http_request_duration_seconds_bucket{job="ml-api"}[5m]))
)

# GPU utilization
avg(nvidia_gpu_utilization{job="ml-api"})

# Model inference latency
histogram_quantile(0.99,
  rate(ml_inference_duration_seconds_bucket{model="anomaly_detector"}[5m])
)

# Prediction throughput
sum(rate(ml_predictions_total[5m]))

# Memory usage
container_memory_usage_bytes{container="ml-api"}
/ container_spec_memory_limit_bytes{container="ml-api"} * 100
```

2.2 Prometheus Configuration

```yaml
# prometheus.yml
global:
  scrape_interval: 15s
  evaluation_interval: 15s
  external_labels:
    cluster: oilgas-production
    environment: production

alerting:
  alertmanagers:
    - static_configs:
        - targets:
            - alertmanager:9093
```

```yaml
rule_files:
  - /etc/prometheus/rules/*.yml

scrape_configs:
  # Prometheus self-monitoring
  - job_name: 'prometheus'
    static_configs:
      - targets: ['localhost:9090']

  # Kubernetes API server
  - job_name: 'kubernetes-apiservers'
    kubernetes_sd_configs:
      - role: endpoints
    scheme: https
    tls_config:
      ca_file: /var/run/secrets/kubernetes.io/serviceaccount/ca.crt
    bearer_token_file: /var/run/secrets/kubernetes.io/serviceaccount/token
    relabel_configs:
      - source_labels: [__meta_kubernetes_namespace, __meta_kubernetes_service_name, __meta_kubernetes_
        action: keep
        regex: default;kubernetes;https

  # Kubernetes nodes
  - job_name: 'kubernetes-nodes'
    kubernetes_sd_configs:
      - role: node
    scheme: https
    tls_config:
      ca_file: /var/run/secrets/kubernetes.io/serviceaccount/ca.crt
    bearer_token_file: /var/run/secrets/kubernetes.io/serviceaccount/token
    relabel_configs:
      - action: labelmap
        regex: __meta_kubernetes_node_label_(.+)

  # Kubernetes pods with prometheus.io annotations
  - job_name: 'kubernetes-pods'
    kubernetes_sd_configs:
      - role: pod
    relabel_configs:
      - source_labels: [__meta_kubernetes_pod_annotation_prometheus_io_scrape]
        action: keep
        regex: true
      - source_labels: [__meta_kubernetes_pod_annotation_prometheus_io_path]
        action: replace
        target_label: __metrics_path__
        regex: (.+)
      - source_labels: [__address__, __meta_kubernetes_pod_annotation_prometheus_io_port]
        action: replace
        regex: ([^:]+)(?::\d+)?;(\d+)
        replacement: $1:$2
        target_label: __address__
      - action: labelmap
        regex: __meta_kubernetes_pod_label_(.+)
```

```
      - source_labels: [__meta_kubernetes_namespace]
        action: replace
        target_label: kubernetes_namespace
      - source_labels: [__meta_kubernetes_pod_name]
        action: replace
        target_label: kubernetes_pod_name

  # ML API specific
  - job_name: 'ml-api'
    kubernetes_sd_configs:
      - role: pod
        namespaces:
          names: ['oilgas-ml']
    relabel_configs:
      - source_labels: [__meta_kubernetes_pod_label_app]
        action: keep
        regex: ml-api
```

2.3 Recording Rules

```
# /etc/prometheus/rules/recording_rules.yml
groups:
  - name: ml-api-recording
    interval: 30s
    rules:
      # Request rate
      - record: ml_api:http_requests:rate5m
        expr: sum(rate(http_requests_total{job="ml-api"}[5m]))

      # Error rate
      - record: ml_api:http_errors:rate5m
        expr: |
          sum(rate(http_requests_total{job="ml-api",status=~"5.."}[5m]))
          / sum(rate(http_requests_total{job="ml-api"}[5m]))

      # P95 latency
      - record: ml_api:http_latency:p95_5m
        expr: |
          histogram_quantile(0.95,
            sum by (le) (rate(http_request_duration_seconds_bucket{job="ml-api"}[5m]))
          )

      # Inference latency by model
      - record: ml_api:inference_latency:p99_5m
        expr: |
          histogram_quantile(0.99,
            sum by (le, model) (rate(ml_inference_duration_seconds_bucket[5m]))
          )

      # GPU memory usage
      - record: ml_api:gpu_memory:usage_ratio
        expr: |
          nvidia_gpu_memory_used_bytes / nvidia_gpu_memory_total_bytes
```

```
      # Prediction throughput
      - record: ml_api:predictions:rate5m
        expr: sum by (model) (rate(ml_predictions_total[5m]))
```

2.4 Prometheus CLI (promtool)

```
# Check configuration
promtool check config prometheus.yml

# Check rules
promtool check rules rules.yml

# Query from command line
promtool query instant http://prometheus:9090 'up{job="ml-api"}'
promtool query range http://prometheus:9090 'rate(http_requests_total[5m])' --start="1h"

# Test rules
promtool test rules test.yml

# TSDB operations
promtool tsdb analyze /prometheus/data
promtool tsdb list /prometheus/data
```

---

3. GRAFANA OPERATIONS

3.1 Grafana CLI & API

```
# ============================================
# GRAFANA CLI
# ============================================

# Login
grafana-cli admin reset-admin-password newpassword

# Install plugins
grafana-cli plugins install grafana-piechart-panel
grafana-cli plugins install grafana-clock-panel
grafana-cli plugins install grafana-worldmap-panel

# List plugins
grafana-cli plugins list-remote
grafana-cli plugins list-versions grafana-piechart-panel

# ============================================
# GRAFANA API
# ============================================

GRAFANA_URL="http://grafana.oilgas.internal:3000"
GRAFANA_API_KEY="your-api-key"

# Get all dashboards
curl -H "Authorization: Bearer $GRAFANA_API_KEY" \
```

```
    "$GRAFANA_URL/api/search?type=dash-db"

# Get dashboard by UID
curl -H "Authorization: Bearer $GRAFANA_API_KEY" \
    "$GRAFANA_URL/api/dashboards/uid/ml-api-dashboard"

# Create/Update dashboard
curl -X POST -H "Authorization: Bearer $GRAFANA_API_KEY" \
    -H "Content-Type: application/json" \
    -d @dashboard.json \
    "$GRAFANA_URL/api/dashboards/db"

# Export dashboard
curl -H "Authorization: Bearer $GRAFANA_API_KEY" \
    "$GRAFANA_URL/api/dashboards/uid/ml-api-dashboard" | jq '.dashboard' > dashboard.json

# Create folder
curl -X POST -H "Authorization: Bearer $GRAFANA_API_KEY" \
    -H "Content-Type: application/json" \
    -d '{"title": "Oil & Gas"}' \
    "$GRAFANA_URL/api/folders"

# Create data source
curl -X POST -H "Authorization: Bearer $GRAFANA_API_KEY" \
    -H "Content-Type: application/json" \
    -d '{
        "name": "Prometheus",
        "type": "prometheus",
        "url": "http://prometheus:9090",
        "access": "proxy",
        "isDefault": true
    }' \
    "$GRAFANA_URL/api/datasources"

# Create alert
curl -X POST -H "Authorization: Bearer $GRAFANA_API_KEY" \
    -H "Content-Type: application/json" \
    -d @alert.json \
    "$GRAFANA_URL/api/ruler/grafana/api/v1/rules/oilgas"
```

3.2 Dashboard JSON Definition

```
{
  "dashboard": {
    "title": "ML API Monitoring",
    "tags": ["ml", "api", "oilgas"],
    "timezone": "browser",
    "refresh": "30s",
    "time": {
      "from": "now-1h",
      "to": "now"
    },
    "panels": [
      {
```

```
    "title": "Request Rate",
    "type": "timeseries",
    "gridPos": {"x": 0, "y": 0, "w": 12, "h": 8},
    "targets": [
      {
        "expr": "sum(rate(http_requests_total{job=\"ml-api\"}[5m])) by (path)",
        "legendFormat": "{{path}}"
      }
    ],
    "fieldConfig": {
      "defaults": {
        "unit": "reqps",
        "custom": {
          "drawStyle": "line",
          "lineInterpolation": "smooth"
        }
      }
    }
  },
  {
    "title": "Error Rate",
    "type": "stat",
    "gridPos": {"x": 12, "y": 0, "w": 6, "h": 4},
    "targets": [
      {
        "expr": "sum(rate(http_requests_total{job=\"ml-api\",status=~\"5..\"}[5m])) / sum(rate(http
        "legendFormat": "Error %"
      }
    ],
    "fieldConfig": {
      "defaults": {
        "unit": "percent",
        "thresholds": {
          "mode": "absolute",
          "steps": [
            {"value": null, "color": "green"},
            {"value": 1, "color": "yellow"},
            {"value": 5, "color": "red"}
          ]
        }
      }
    }
  },
  {
    "title": "P95 Latency",
    "type": "gauge",
    "gridPos": {"x": 18, "y": 0, "w": 6, "h": 4},
    "targets": [
      {
        "expr": "histogram_quantile(0.95, sum by (le) (rate(http_request_duration_seconds_bucket{jo
        "legendFormat": "P95"
      }
    ],
    "fieldConfig": {
```

```
    "defaults": {
      "unit": "s",
      "min": 0,
      "max": 1,
      "thresholds": {
        "mode": "absolute",
        "steps": [
          {"value": null, "color": "green"},
          {"value": 0.2, "color": "yellow"},
          {"value": 0.5, "color": "red"}
        ]
      }
    }
  }
},
{
  "title": "GPU Utilization",
  "type": "timeseries",
  "gridPos": {"x": 0, "y": 8, "w": 12, "h": 8},
  "targets": [
    {
      "expr": "avg(nvidia_gpu_utilization{job=\"ml-api\"}) by (gpu)",
      "legendFormat": "GPU {{gpu}}"
    }
  ],
  "fieldConfig": {
    "defaults": {
      "unit": "percent",
      "min": 0,
      "max": 100
    }
  }
},
{
  "title": "Inference Latency by Model",
  "type": "timeseries",
  "gridPos": {"x": 12, "y": 8, "w": 12, "h": 8},
  "targets": [
    {
      "expr": "histogram_quantile(0.99, sum by (le, model) (rate(ml_inference_duration_seconds_bu
      "legendFormat": "{{model}} P99"
    }
  ],
  "fieldConfig": {
    "defaults": {
      "unit": "s"
    }
  }
},
{
  "title": "Predictions by Model",
  "type": "piechart",
  "gridPos": {"x": 0, "y": 16, "w": 6, "h": 8},
  "targets": [
```

```
          {
            "expr": "sum by (model) (increase(ml_predictions_total[1h]))",
            "legendFormat": "{{model}}"
          }
        ]
      }
    ],
    "templating": {
      "list": [
        {
          "name": "namespace",
          "type": "query",
          "query": "label_values(kube_pod_info, namespace)",
          "current": {"text": "oilgas-ml", "value": "oilgas-ml"}
        },
        {
          "name": "model",
          "type": "query",
          "query": "label_values(ml_predictions_total{namespace=\"$namespace\"}, model)",
          "multi": true,
          "includeAll": true
        }
      ]
    }
  },
  "folderId": 0,
  "overwrite": true
}
```

---

4. ELK STACK OPERATIONS

4.1 Elasticsearch Operations

```
# ============================================
# CLUSTER HEALTH & INFO
# ============================================

ES_URL="http://elasticsearch.oilgas.internal:9200"

# Cluster health
curl "$ES_URL/_cluster/health?pretty"

# Cluster stats
curl "$ES_URL/_cluster/stats?pretty"

# Node info
curl "$ES_URL/_cat/nodes?v"

# Indices
curl "$ES_URL/_cat/indices?v"
curl "$ES_URL/_cat/indices?v&s=store.size:desc"
```

```
# ==========================================
# INDEX MANAGEMENT
# ==========================================

# Create index with mapping
curl -X PUT "$ES_URL/ml-api-logs" -H 'Content-Type: application/json' -d'
{
  "settings": {
    "number_of_shards": 3,
    "number_of_replicas": 1,
    "index.lifecycle.name": "logs-policy",
    "index.lifecycle.rollover_alias": "ml-api-logs"
  },
  "mappings": {
    "properties": {
      "@timestamp": {"type": "date"},
      "level": {"type": "keyword"},
      "message": {"type": "text"},
      "service": {"type": "keyword"},
      "trace_id": {"type": "keyword"},
      "span_id": {"type": "keyword"},
      "request_id": {"type": "keyword"},
      "model": {"type": "keyword"},
      "latency_ms": {"type": "float"},
      "prediction": {"type": "object"}
    }
  }
}'

# Delete index
curl -X DELETE "$ES_URL/old-index"

# Reindex
curl -X POST "$ES_URL/_reindex" -H 'Content-Type: application/json' -d'
{
  "source": {"index": "old-index"},
  "dest": {"index": "new-index"}
}'

# ==========================================
# SEARCH QUERIES
# ==========================================

# Simple search
curl "$ES_URL/ml-api-logs/_search?q=level:ERROR&pretty"

# Query DSL
curl -X POST "$ES_URL/ml-api-logs/_search?pretty" -H 'Content-Type: application/json' -d'
{
  "query": {
    "bool": {
      "must": [
        {"match": {"service": "ml-api"}},
        {"range": {"@timestamp": {"gte": "now-1h"}}}
```

```
      ],
      "filter": [
        {"term": {"level": "ERROR"}}
      ]
    }
  },
  "sort": [{"@timestamp": "desc"}],
  "size": 100
}'

# Aggregations
curl -X POST "$ES_URL/ml-api-logs/_search?pretty" -H 'Content-Type: application/json' -d'
{
  "size": 0,
  "query": {"range": {"@timestamp": {"gte": "now-24h"}}},
  "aggs": {
    "errors_by_service": {
      "terms": {"field": "service"},
      "aggs": {
        "error_count": {
          "filter": {"term": {"level": "ERROR"}}
        }
      }
    },
    "latency_percentiles": {
      "percentiles": {"field": "latency_ms"}
    },
    "errors_over_time": {
      "date_histogram": {
        "field": "@timestamp",
        "fixed_interval": "1h"
      },
      "aggs": {
        "error_count": {"filter": {"term": {"level": "ERROR"}}}
      }
    }
  }
}'

# ==========================================
# INDEX LIFECYCLE MANAGEMENT (ILM)
# ==========================================

# Create ILM policy
curl -X PUT "$ES_URL/_ilm/policy/logs-policy" -H 'Content-Type: application/json' -d'
{
  "policy": {
    "phases": {
      "hot": {
        "min_age": "0ms",
        "actions": {
          "rollover": {
            "max_primary_shard_size": "50GB",
            "max_age": "1d"
```

```
          }
        }
      },
      "warm": {
        "min_age": "7d",
        "actions": {
          "shrink": {"number_of_shards": 1},
          "forcemerge": {"max_num_segments": 1}
        }
      },
      "cold": {
        "min_age": "30d",
        "actions": {
          "freeze": {}
        }
      },
      "delete": {
        "min_age": "90d",
        "actions": {
          "delete": {}
        }
      }
    }
  }
}'
```

4.2 Filebeat Configuration

```yaml
# filebeat.yml
filebeat.inputs:
  # Container logs
  - type: container
    paths:
      - '/var/lib/docker/containers/*/*.log'
    processors:
      - add_kubernetes_metadata:
          host: ${NODE_NAME}
          matchers:
            - logs_path:
                logs_path: "/var/lib/docker/containers/"

  # Application logs
  - type: log
    enabled: true
    paths:
      - /var/log/ml-api/*.log
    json.keys_under_root: true
    json.overwrite_keys: true
    json.add_error_key: true
    fields:
      service: ml-api
    fields_under_root: true

processors:
```

```
  - add_host_metadata: ~
  - add_cloud_metadata: ~
  - add_docker_metadata: ~
  - add_kubernetes_metadata: ~

  # Parse JSON logs
  - decode_json_fields:
      fields: ["message"]
      target: ""
      overwrite_keys: true

  # Drop debug logs in production
  - drop_event:
      when:
        equals:
          level: "DEBUG"

  # Add fields based on log level
  - add_fields:
      target: alert
      fields:
        severity: "low"
      when:
        equals:
          level: "WARNING"

  - add_fields:
      target: alert
      fields:
        severity: "high"
      when:
        equals:
          level: "ERROR"

output.elasticsearch:
  hosts: ["elasticsearch:9200"]
  index: "ml-api-logs-%{+yyyy.MM.dd}"
  pipeline: ml-api-pipeline

setup.template:
  name: "ml-api-logs"
  pattern: "ml-api-logs-*"

setup.ilm:
  enabled: true
  rollover_alias: "ml-api-logs"
  policy_name: "logs-policy"

monitoring:
  enabled: true
  elasticsearch:
    hosts: ["elasticsearch:9200"]
```

4.3 Logstash Pipeline

```
# logstash.conf
input {
  beats {
    port => 5044
  }

  kafka {
    bootstrap_servers => "kafka:9092"
    topics => ["ml-api-logs"]
    codec => json
  }
}

filter {
  # Parse timestamp
  date {
    match => ["timestamp", "ISO8601", "yyyy-MM-dd HH:mm:ss"]
    target => "@timestamp"
  }

  # Parse JSON message if string
  if [message] =~ /^\{.*\}$/ {
    json {
      source => "message"
      target => "parsed"
    }
    mutate {
      rename => { "[parsed][level]" => "level" }
      rename => { "[parsed][msg]" => "message" }
      rename => { "[parsed][trace_id]" => "trace_id" }
    }
  }

  # Grok for unstructured logs
  if ![level] {
    grok {
      match => { "message" => "%{TIMESTAMP_ISO8601:timestamp} %{LOGLEVEL:level} %{GREEDYDATA:message}" }
      overwrite => ["message"]
    }
  }

  # Extract model and inference info
  if [message] =~ /inference/ {
    grok {
      match => { "message" => "model=%{WORD:model} latency=%{NUMBER:latency_ms:float}ms" }
    }
  }

  # GeoIP for client IPs
  if [client_ip] {
    geoip {
      source => "client_ip"
```

```
      target => "geoip"
    }
  }

  # Add severity based on level
  if [level] == "ERROR" or [level] == "CRITICAL" {
    mutate {
      add_field => { "alert_severity" => "high" }
    }
  } else if [level] == "WARNING" {
    mutate {
      add_field => { "alert_severity" => "medium" }
    }
  } else {
    mutate {
      add_field => { "alert_severity" => "low" }
    }
  }

  # Remove unnecessary fields
  mutate {
    remove_field => ["agent", "ecs", "input", "host"]
  }
}

output {
  elasticsearch {
    hosts => ["elasticsearch:9200"]
    index => "ml-api-logs-%{+YYYY.MM.dd}"
    document_id => "%{[@metadata][_id]}"
  }

  # Send errors to alert channel
  if [level] == "ERROR" {
    http {
      url => "http://alertmanager:9093/api/v1/alerts"
      http_method => "post"
      format => "json"
      content_type => "application/json"
      mapping => {
        "labels" => {
          "alertname" => "ApplicationError"
          "service" => "%{service}"
          "severity" => "%{alert_severity}"
        }
        "annotations" => {
          "summary" => "Application error in %{service}"
          "description" => "%{message}"
        }
      }
    }
  }
}
```

5. ALERTMANAGER CONFIGURATION

5.1 Alerting Rules

```yaml
# /etc/prometheus/rules/alerting_rules.yml
groups:
  - name: ml-api-alerts
    rules:
      # High error rate
      - alert: MLAPIHighErrorRate
        expr: |
          sum(rate(http_requests_total{job="ml-api",status=~"5.."}[5m]))
          / sum(rate(http_requests_total{job="ml-api"}[5m])) > 0.05
        for: 5m
        labels:
          severity: critical
          team: ml
        annotations:
          summary: "High error rate on ML API"
          description: "Error rate is {{ $value | humanizePercentage }} (>5%)"
          runbook_url: "https://wiki.oilgas.internal/runbooks/ml-api-errors"

      # High latency
      - alert: MLAPIHighLatency
        expr: |
          histogram_quantile(0.95,
            sum by (le) (rate(http_request_duration_seconds_bucket{job="ml-api"}[5m]))
          ) > 0.5
        for: 5m
        labels:
          severity: warning
          team: ml
        annotations:
          summary: "High latency on ML API"
          description: "P95 latency is {{ $value | humanizeDuration }}"

      # Pod not ready
      - alert: MLAPIPodNotReady
        expr: |
          kube_pod_status_ready{namespace="oilgas-ml", pod=~"ml-api.*"} == 0
        for: 5m
        labels:
          severity: warning
          team: platform
        annotations:
          summary: "ML API pod not ready"
          description: "Pod {{ $labels.pod }} is not ready"

      # GPU memory high
      - alert: MLAPIGPUMemoryHigh
        expr: |
          nvidia_gpu_memory_used_bytes / nvidia_gpu_memory_total_bytes > 0.9
```

```
    for: 10m
    labels:
      severity: warning
      team: ml
    annotations:
      summary: "High GPU memory usage"
      description: "GPU {{ $labels.gpu }} memory usage is {{ $value | humanizePercentage }}"

  # Model inference slow
  - alert: ModelInferenceSlow
    expr: |
      histogram_quantile(0.99,
        sum by (le, model) (rate(ml_inference_duration_seconds_bucket[5m]))
      ) > 1
    for: 5m
    labels:
      severity: warning
      team: ml
    annotations:
      summary: "Slow model inference"
      description: "Model {{ $labels.model }} P99 latency is {{ $value | humanizeDuration }}"

  # No predictions
  - alert: NoPredictions
    expr: |
      sum(rate(ml_predictions_total[5m])) == 0
    for: 10m
    labels:
      severity: critical
      team: ml
    annotations:
      summary: "No predictions being made"
      description: "ML API has not made any predictions in the last 10 minutes"

  # Disk space
  - alert: DiskSpaceLow
    expr: |
      (node_filesystem_avail_bytes{mountpoint="/"} / node_filesystem_size_bytes{mountpoint="/"}) < (
    for: 5m
    labels:
      severity: critical
      team: platform
    annotations:
      summary: "Low disk space on {{ $labels.instance }}"
      description: "Disk space is {{ $value | humanizePercentage }} available"
```

5.2 Alertmanager Configuration

```
# alertmanager.yml
global:
  resolve_timeout: 5m
  smtp_smarthost: 'smtp.oilgas.internal:587'
  smtp_from: 'alertmanager@oilgas.internal'
  slack_api_url: 'https://hooks.slack.com/services/xxx/yyy/zzz'
```

```yaml
  pagerduty_url: 'https://events.pagerduty.com/v2/enqueue'

templates:
  - '/etc/alertmanager/templates/*.tmpl'

route:
  group_by: ['alertname', 'cluster', 'service']
  group_wait: 30s
  group_interval: 5m
  repeat_interval: 4h
  receiver: 'default-receiver'
  routes:
    # Critical alerts -> PagerDuty
    - match:
        severity: critical
      receiver: 'pagerduty-critical'
      continue: true

    # ML team alerts
    - match:
        team: ml
      receiver: 'ml-team-slack'
      routes:
        - match:
            severity: critical
          receiver: 'ml-team-pagerduty'

    # Platform team alerts
    - match:
        team: platform
      receiver: 'platform-team-slack'

inhibit_rules:
  - source_match:
      severity: 'critical'
    target_match:
      severity: 'warning'
    equal: ['alertname', 'cluster', 'service']

receivers:
  - name: 'default-receiver'
    slack_configs:
      - channel: '#alerts'
        send_resolved: true
        title: '{{ template "slack.title" . }}'
        text: '{{ template "slack.text" . }}'

  - name: 'ml-team-slack'
    slack_configs:
      - channel: '#ml-alerts'
        send_resolved: true
        color: '{{ if eq .Status "firing" }}danger{{ else }}good{{ end }}'
        title: '[{{ .Status | toUpper }}] {{ .CommonLabels.alertname }}'
        text: |
```

```
        *Summary:* {{ .CommonAnnotations.summary }}
        *Description:* {{ .CommonAnnotations.description }}
        *Severity:* {{ .CommonLabels.severity }}
        {{ if .CommonAnnotations.runbook_url }}*Runbook:* {{ .CommonAnnotations.runbook_url }}{{ end }}

  - name: 'ml-team-pagerduty'
    pagerduty_configs:
      - service_key: 'your-pagerduty-key'
        severity: '{{ .CommonLabels.severity }}'
        description: '{{ .CommonAnnotations.summary }}'
        details:
          firing: '{{ .Alerts.Firing | len }}'
          resolved: '{{ .Alerts.Resolved | len }}'

  - name: 'pagerduty-critical'
    pagerduty_configs:
      - service_key: 'critical-pagerduty-key'
        severity: critical

  - name: 'platform-team-slack'
    slack_configs:
      - channel: '#platform-alerts'
        send_resolved: true
```

---

6. JAEGER TRACING

6.1 Jaeger Query CLI

```
# Get services
curl "http://jaeger.oilgas.internal:16686/api/services" | jq

# Get operations for service
curl "http://jaeger.oilgas.internal:16686/api/services/ml-api/operations" | jq

# Search traces
curl "http://jaeger.oilgas.internal:16686/api/traces?service=ml-api&limit=20&lookback=1h" | jq

# Get trace by ID
curl "http://jaeger.oilgas.internal:16686/api/traces/abc123" | jq

# Search with tags
curl "http://jaeger.oilgas.internal:16686/api/traces?service=ml-api&tags=%7B%22http.status_code%22%3A%2
```

6.2 Python Tracing Integration

```
#!/usr/bin/env python3
# utils/tracing.py
"""
Distributed tracing with OpenTelemetry/Jaeger

Usage:
    from utils.tracing import init_tracing, trace_function
```

```
    init_tracing("ml-api")

    @trace_function
    def my_function():
        pass
"""

from opentelemetry import trace
from opentelemetry.exporter.jaeger.thrift import JaegerExporter
from opentelemetry.sdk.trace import TracerProvider
from opentelemetry.sdk.trace.export import BatchSpanProcessor
from opentelemetry.sdk.resources import Resource
from opentelemetry.instrumentation.fastapi import FastAPIInstrumentor
from opentelemetry.instrumentation.requests import RequestsInstrumentor
from opentelemetry.instrumentation.sqlalchemy import SQLAlchemyInstrumentor
from functools import wraps
import os


def init_tracing(service_name: str):
    """Initialize OpenTelemetry tracing"""
    resource = Resource.create({
        "service.name": service_name,
        "service.version": os.environ.get("VERSION", "unknown"),
        "deployment.environment": os.environ.get("ENVIRONMENT", "development")
    })

    provider = TracerProvider(resource=resource)

    # Jaeger exporter
    jaeger_exporter = JaegerExporter(
        agent_host_name=os.environ.get("JAEGER_AGENT_HOST", "localhost"),
        agent_port=int(os.environ.get("JAEGER_AGENT_PORT", 6831))
    )

    provider.add_span_processor(BatchSpanProcessor(jaeger_exporter))
    trace.set_tracer_provider(provider)

    # Auto-instrument libraries
    RequestsInstrumentor().instrument()

    return trace.get_tracer(service_name)


def instrument_fastapi(app):
    """Instrument FastAPI application"""
    FastAPIInstrumentor.instrument_app(app)


def instrument_sqlalchemy(engine):
    """Instrument SQLAlchemy"""
    SQLAlchemyInstrumentor().instrument(engine=engine)
```

```python
def trace_function(func):
    """Decorator to trace a function"""
    tracer = trace.get_tracer(__name__)

    @wraps(func)
    def wrapper(*args, **kwargs):
        with tracer.start_as_current_span(func.__name__) as span:
            span.set_attribute("function.name", func.__name__)
            span.set_attribute("function.module", func.__module__)

            try:
                result = func(*args, **kwargs)
                span.set_status(trace.Status(trace.StatusCode.OK))
                return result
            except Exception as e:
                span.set_status(trace.Status(trace.StatusCode.ERROR, str(e)))
                span.record_exception(e)
                raise

    return wrapper


class TracingMiddleware:
    """Custom tracing middleware for ML inference"""

    def __init__(self, app, tracer):
        self.app = app
        self.tracer = tracer

    async def __call__(self, scope, receive, send):
        if scope["type"] != "http":
            await self.app(scope, receive, send)
            return

        with self.tracer.start_as_current_span("http_request") as span:
            span.set_attribute("http.method", scope.get("method", ""))
            span.set_attribute("http.url", scope.get("path", ""))

            # Add custom attributes for ML
            if "predict" in scope.get("path", ""):
                span.set_attribute("ml.operation", "inference")

            await self.app(scope, receive, send)
```

---

7. APPLICATION METRICS

7.1 Python Metrics Integration

```python
#!/usr/bin/env python3
# utils/metrics.py
"""
Prometheus metrics for ML API
```

```
Usage:
    from utils.metrics import metrics
    metrics.inference_counter.labels(model="anomaly").inc()
    metrics.inference_latency.labels(model="anomaly").observe(0.1)
"""

from prometheus_client import Counter, Histogram, Gauge, Info, CollectorRegistry, generate_latest
from prometheus_client.multiprocess import MultiProcessCollector
import os


class MetricsCollector:
    """Application metrics collector"""

    def __init__(self, app_name: str = "ml_api"):
        self.app_name = app_name

        # Check if running in multiprocess mode
        if 'prometheus_multiproc_dir' in os.environ:
            self.registry = CollectorRegistry()
            MultiProcessCollector(self.registry)
        else:
            self.registry = CollectorRegistry()

        self._init_metrics()

    def _init_metrics(self):
        """Initialize all metrics"""

        # HTTP metrics
        self.http_requests = Counter(
            'http_requests_total',
            'Total HTTP requests',
            ['method', 'path', 'status'],
            registry=self.registry
        )

        self.http_request_duration = Histogram(
            'http_request_duration_seconds',
            'HTTP request latency',
            ['method', 'path'],
            buckets=[0.01, 0.05, 0.1, 0.25, 0.5, 1.0, 2.5, 5.0, 10.0],
            registry=self.registry
        )

        self.http_requests_in_progress = Gauge(
            'http_requests_in_progress',
            'HTTP requests currently being processed',
            ['method', 'path'],
            registry=self.registry
        )

        # ML inference metrics
```

```
        self.inference_counter = Counter(
            'ml_predictions_total',
            'Total ML predictions',
            ['model', 'prediction_type'],
            registry=self.registry
        )

        self.inference_latency = Histogram(
            'ml_inference_duration_seconds',
            'ML inference latency',
            ['model'],
            buckets=[0.001, 0.005, 0.01, 0.025, 0.05, 0.1, 0.25, 0.5, 1.0],
            registry=self.registry
        )

        self.inference_errors = Counter(
            'ml_inference_errors_total',
            'ML inference errors',
            ['model', 'error_type'],
            registry=self.registry
        )

        self.batch_size = Histogram(
            'ml_inference_batch_size',
            'Inference batch size',
            ['model'],
            buckets=[1, 2, 4, 8, 16, 32, 64, 128, 256],
            registry=self.registry
        )

        # Model metrics
        self.model_loaded = Gauge(
            'ml_model_loaded',
            'Model loaded status',
            ['model', 'version'],
            registry=self.registry
        )

        self.model_memory_bytes = Gauge(
            'ml_model_memory_bytes',
            'Model memory usage',
            ['model'],
            registry=self.registry
        )

        # GPU metrics (if available)
        self.gpu_utilization = Gauge(
            'gpu_utilization_percent',
            'GPU utilization percentage',
            ['gpu_id'],
            registry=self.registry
        )

        self.gpu_memory_used = Gauge(
```

```
        'gpu_memory_used_bytes',
        'GPU memory used',
        ['gpu_id'],
        registry=self.registry
    )

    self.gpu_memory_total = Gauge(
        'gpu_memory_total_bytes',
        'GPU total memory',
        ['gpu_id'],
        registry=self.registry
    )

    # RAG metrics
    self.rag_queries = Counter(
        'rag_queries_total',
        'Total RAG queries',
        ['collection'],
        registry=self.registry
    )

    self.rag_latency = Histogram(
        'rag_query_duration_seconds',
        'RAG query latency',
        ['stage'],  # embed, search, rerank, generate
        buckets=[0.01, 0.05, 0.1, 0.25, 0.5, 1.0, 2.5, 5.0],
        registry=self.registry
    )

    self.rag_documents_retrieved = Histogram(
        'rag_documents_retrieved',
        'Number of documents retrieved',
        ['collection'],
        buckets=[1, 5, 10, 20, 50, 100],
        registry=self.registry
    )

    self.cache_hits = Counter(
        'cache_hits_total',
        'Cache hits',
        ['cache_type'],  # query, embedding, response
        registry=self.registry
    )

    self.cache_misses = Counter(
        'cache_misses_total',
        'Cache misses',
        ['cache_type'],
        registry=self.registry
    )

    # Application info
    self.app_info = Info(
        'app_info',
```

```
            'Application information',
            registry=self.registry
        )

    def set_app_info(self, version: str, commit: str):
        """Set application info"""
        self.app_info.info({
            'version': version,
            'commit': commit,
            'app': self.app_name
        })

    def get_metrics(self) -> bytes:
        """Get metrics in Prometheus format"""
        return generate_latest(self.registry)


# Global metrics instance
metrics = MetricsCollector()


# FastAPI middleware
from starlette.middleware.base import BaseHTTPMiddleware
from starlette.requests import Request
import time


class PrometheusMiddleware(BaseHTTPMiddleware):
    """Prometheus metrics middleware for FastAPI"""

    async def dispatch(self, request: Request, call_next):
        method = request.method
        path = request.url.path

        # Skip metrics endpoint
        if path == "/metrics":
            return await call_next(request)

        # Track in-progress requests
        metrics.http_requests_in_progress.labels(method=method, path=path).inc()

        start_time = time.perf_counter()
        try:
            response = await call_next(request)
            status = response.status_code
        except Exception as e:
            status = 500
            raise
        finally:
            duration = time.perf_counter() - start_time

            metrics.http_requests.labels(
                method=method, path=path, status=str(status)
            ).inc()
```

```
            metrics.http_request_duration.labels(
                method=method, path=path
            ).observe(duration)

            metrics.http_requests_in_progress.labels(
                method=method, path=path
            ).dec()

        return response
```

7.2 FastAPI Metrics Endpoint

```
# main.py
from fastapi import FastAPI, Response
from utils.metrics import metrics, PrometheusMiddleware

app = FastAPI()

# Add metrics middleware
app.add_middleware(PrometheusMiddleware)

# Set app info
metrics.set_app_info(
    version=os.environ.get("VERSION", "1.0.0"),
    commit=os.environ.get("GIT_COMMIT", "unknown")
)


@app.get("/metrics")
def get_metrics():
    """Prometheus metrics endpoint"""
    return Response(
        content=metrics.get_metrics(),
        media_type="text/plain"
    )


@app.get("/health")
def health_check():
    """Health check endpoint"""
    return {"status": "healthy"}


@app.post("/predict/anomaly")
async def predict_anomaly(request: PredictionRequest):
    """Anomaly detection endpoint with metrics"""
    import time

    start_time = time.perf_counter()

    try:
        result = model.predict(request.features)
```

```
        # Record metrics
        duration = time.perf_counter() - start_time
        metrics.inference_counter.labels(model="anomaly", prediction_type="anomaly").inc()
        metrics.inference_latency.labels(model="anomaly").observe(duration)
        metrics.batch_size.labels(model="anomaly").observe(len(request.features))

        return result

    except Exception as e:
        metrics.inference_errors.labels(model="anomaly", error_type=type(e).__name__).inc()
        raise
```

---

8. LOG MANAGEMENT

8.1 Structured Logging

```
#!/usr/bin/env python3
# utils/logging.py
"""
Structured logging configuration

Usage:
    from utils.logging import setup_logging, get_logger
    setup_logging()
    logger = get_logger(__name__)
    logger.info("Processing request", request_id="abc123", model="anomaly")
"""

import sys
import json
import logging
from datetime import datetime
from typing import Any, Dict
import structlog
from pythonjsonlogger import jsonlogger


def setup_logging(
    level: str = "INFO",
    json_output: bool = True,
    service_name: str = "ml-api"
):
    """Configure structured logging"""

    if json_output:
        # JSON formatter for production
        structlog.configure(
            processors=[
                structlog.stdlib.filter_by_level,
                structlog.stdlib.add_logger_name,
                structlog.stdlib.add_log_level,
                structlog.stdlib.PositionalArgumentsFormatter(),
```

```
                structlog.processors.TimeStamper(fmt="iso"),
                structlog.processors.StackInfoRenderer(),
                structlog.processors.format_exc_info,
                structlog.processors.UnicodeDecoder(),
                structlog.processors.JSONRenderer()
            ],
            context_class=dict,
            logger_factory=structlog.stdlib.LoggerFactory(),
            wrapper_class=structlog.stdlib.BoundLogger,
            cache_logger_on_first_use=True,
        )

        # Configure standard logging
        handler = logging.StreamHandler(sys.stdout)
        handler.setFormatter(jsonlogger.JsonFormatter(
            fmt='%(timestamp)s %(level)s %(name)s %(message)s'
        ))
        logging.root.handlers = [handler]
        logging.root.setLevel(getattr(logging, level))

    else:
        # Human-readable format for development
        structlog.configure(
            processors=[
                structlog.stdlib.filter_by_level,
                structlog.stdlib.add_log_level,
                structlog.dev.ConsoleRenderer()
            ],
            context_class=dict,
            logger_factory=structlog.stdlib.LoggerFactory(),
            wrapper_class=structlog.stdlib.BoundLogger,
            cache_logger_on_first_use=True,
        )

        logging.basicConfig(
            format="%(asctime)s %(levelname)s %(name)s: %(message)s",
            level=getattr(logging, level),
            stream=sys.stdout
        )

    # Add service context
    structlog.contextvars.bind_contextvars(
        service=service_name,
        version=os.environ.get("VERSION", "unknown")
    )


def get_logger(name: str):
    """Get a structured logger"""
    return structlog.get_logger(name)


class RequestContextMiddleware:
    """Middleware to add request context to logs"""
```

```
    def __init__(self, app):
        self.app = app

    async def __call__(self, scope, receive, send):
        if scope["type"] != "http":
            await self.app(scope, receive, send)
            return

        import uuid

        # Generate request ID
        request_id = scope.get("headers", {}).get(
            b"x-request-id", str(uuid.uuid4()).encode()
        ).decode()

        # Bind to log context
        structlog.contextvars.bind_contextvars(
            request_id=request_id,
            path=scope.get("path"),
            method=scope.get("method")
        )

        try:
            await self.app(scope, receive, send)
        finally:
            structlog.contextvars.unbind_contextvars(
                "request_id", "path", "method"
            )


# Example usage
logger = get_logger(__name__)

# Log with context
logger.info(
    "Processing inference request",
    model="anomaly_detector",
    batch_size=32,
    input_shape=[32, 15]
)

# Log with exception
try:
    result = model.predict(data)
except Exception as e:
    logger.error(
        "Inference failed",
        model="anomaly_detector",
        error=str(e),
        exc_info=True
    )
```

9. DASHBOARDS & VISUALIZATION

9.1 Grafana Dashboard Provisioning

```yaml
# grafana/provisioning/dashboards/dashboards.yaml
apiVersion: 1

providers:
  - name: 'default'
    orgId: 1
    folder: 'Oil & Gas'
    type: file
    disableDeletion: false
    updateIntervalSeconds: 10
    allowUiUpdates: true
    options:
      path: /var/lib/grafana/dashboards
---
# grafana/provisioning/datasources/datasources.yaml
apiVersion: 1

datasources:
  - name: Prometheus
    type: prometheus
    access: proxy
    url: http://prometheus:9090
    isDefault: true
    editable: false

  - name: Elasticsearch
    type: elasticsearch
    access: proxy
    url: http://elasticsearch:9200
    database: "ml-api-logs-*"
    jsonData:
      timeField: "@timestamp"
      esVersion: "8.0.0"
      logMessageField: message
      logLevelField: level

  - name: Jaeger
    type: jaeger
    access: proxy
    url: http://jaeger:16686
```

---

10. COMPLETE MONITORING WORKFLOWS

10.1 Setup Script

```bash
#!/bin/bash
# workflows/setup_monitoring.sh
# Deploy complete monitoring stack
```

```bash
set -e

NAMESPACE="monitoring"

echo "Setting up monitoring stack"
echo "============================"

# Create namespace
kubectl create namespace $NAMESPACE --dry-run=client -o yaml | kubectl apply -f -

# Deploy Prometheus Stack
echo "Deploying Prometheus..."
helm upgrade --install prometheus prometheus-community/kube-prometheus-stack \
    -n $NAMESPACE \
    -f monitoring/prometheus-values.yaml

# Deploy ELK Stack
echo "Deploying Elasticsearch..."
helm upgrade --install elasticsearch elastic/elasticsearch \
    -n $NAMESPACE \
    -f monitoring/elasticsearch-values.yaml

echo "Deploying Kibana..."
helm upgrade --install kibana elastic/kibana \
    -n $NAMESPACE \
    -f monitoring/kibana-values.yaml

echo "Deploying Filebeat..."
helm upgrade --install filebeat elastic/filebeat \
    -n $NAMESPACE \
    -f monitoring/filebeat-values.yaml

# Deploy Jaeger
echo "Deploying Jaeger..."
helm upgrade --install jaeger jaegertracing/jaeger \
    -n $NAMESPACE \
    -f monitoring/jaeger-values.yaml

# Wait for deployments
echo "Waiting for deployments..."
kubectl rollout status deployment/prometheus-grafana -n $NAMESPACE
kubectl rollout status statefulset/elasticsearch-master -n $NAMESPACE

# Import Grafana dashboards
echo "Importing Grafana dashboards..."
for dashboard in monitoring/dashboards/*.json; do
    curl -X POST "http://admin:admin@grafana.$NAMESPACE:3000/api/dashboards/db" \
        -H "Content-Type: application/json" \
        -d @"$dashboard"
done

echo ""
echo "Monitoring stack deployed!"
echo "Grafana: http://grafana.$NAMESPACE:3000"
```

```
echo "Prometheus: http://prometheus.$NAMESPACE:9090"
echo "Kibana: http://kibana.$NAMESPACE:5601"
echo "Jaeger: http://jaeger.$NAMESPACE:16686"
```

10.2 Quick Reference

```
# ============================================
# MONITORING QUICK REFERENCE
# ============================================

# --- PROMETHEUS ---
# Query metrics
curl 'http://prometheus:9090/api/v1/query?query=up'

# Check rules
promtool check rules rules.yml

# --- GRAFANA ---
# List dashboards
curl -H "Authorization: Bearer $API_KEY" "http://grafana:3000/api/search"

# Export dashboard
curl -H "Authorization: Bearer $API_KEY" "http://grafana:3000/api/dashboards/uid/xyz"

# --- ELASTICSEARCH ---
# Cluster health
curl "http://elasticsearch:9200/_cluster/health?pretty"

# Search logs
curl "http://elasticsearch:9200/ml-api-logs-*/_search?q=level:ERROR"

# --- ALERTMANAGER ---
# List alerts
curl "http://alertmanager:9093/api/v1/alerts"

# Silence alert
curl -X POST "http://alertmanager:9093/api/v1/silences" -d @silence.json

# --- JAEGER ---
# Get services
curl "http://jaeger:16686/api/services"

# Search traces
curl "http://jaeger:16686/api/traces?service=ml-api&limit=10"
```

END OF MONITORING TOOLS & OPERATIONS GUIDE